CSCI-UA 102 — Quiz 12

Graph traversals

Name: _____ NetID: _____

You may assume the following interface and helpers:

```
public interface Graph<V, E> {
    PositionList<Vertex<V>> vertices();
    PositionList<Edge<E, V>> edges();

    PositionList<Edge<E, V>> outgoingEdges(Vertex<V> v);
    PositionList<Edge<E, V>> incomingEdges(Vertex<V> v);

    Edge<E, V> getEdge(Vertex<V> from, Vertex<V> to);
    Vertex<V>[] endVertices(Edge<E, V> e);
    Vertex<V> opposite(Vertex<V> v, Edge<E, V> e);

    void insertVertex(V x);
    void insertEdge(Vertex<V> from, Vertex<V> to, E x);
    void removeVertex(Vertex<V> v);
    void removeEdge(Edge<E, V> e);

    int numVertices();
    int numEdges();
    int outDegree(Vertex<V> v);
    int inDegree(Vertex<V> v);
}
```

1. (3 points) Write a method that returns a list of edges making up a directed spanning tree of the descendants of `v`. You may use a `HashSet<Vertex<V>>` to track visited vertices.

```
static <E, V> PositionList<Edge<E, V>> spanningTree(
        Graph<V, E> graph, Vertex<V> v)
```

2. (2 points) Write a method that takes in a list of spanning edges of descendants of `v` and a descendant of `v`, `u`, and returns a list of vertices making up a directed path from `v` to `u`. Don't worry about computational complexity.

```
static <E, V> PositionList<Vertex<V>> path(
        PositionList<Edge<E, V>> spanningEdges, Vertex<V> u)
```

Bonus (1 point): What is the worst-case asymptotic computational complexity of your `path` implementation in terms of n, the number of descendants of `v`? How could you make it more efficient?

1. spanningTree (iterative DFS, like in class)

```
static <E, V> PositionList<Edge<E, V>> spanningTree(
        Graph<V, E> graph, Vertex<V> v) {
    PositionList<Edge<E, V>> result = new DoublyLinkedList<>();
    Stack<Edge<E, V>> stack = new LinkedStack<>();
    Map<Vertex<V>, Object> seen = new HashMapSC<>();
    seen.put(v, 0);
    for (Edge<E, V> edge : graph.outgoingEdges(v)) {
        stack.push(edge);
    }
    while (stack.size() > 0) {
        Edge<E, V> e = stack.pop();
        Vertex<V> w = e.getEndpoints()[1];
        if (seen.get(w) == null) {
            result.addLast(e);
            for (Edge<E, V> edge : graph.outgoingEdges(w)) {
                stack.push(edge);
            }
            seen.put(w, 0);
        }
    }
    return result;
}
```

2. path

```
static <E, V> PositionList<Vertex<V>> path(
        PositionList<Edge<E, V>> spanningEdges, Vertex<V> u) {
    PositionList<Vertex<V>> result = new DoublyLinkedList<>();
    result.addFirst(u);
    Vertex<V> cur = u;
    while (true) {
        Vertex<V> parent = null;
        for (Edge<E, V> e : spanningEdges) {
            Vertex<V>[] ends = e.getEndpoints();
            if (ends[1] == cur) {
                parent = ends[0];
                break;
            }
        }
        if (parent == null) break; // cur is the root v
        result.addFirst(parent);
        cur = parent;
    }
    return result;
}
```

Bonus

The path has at most n vertices. Each step scans the edge list of size $n-1$ to find the parent, so worst case is $O(n^2)$. You could build a `HashMapSC<Vertex<V>, Vertex<V>>` mapping each child to its parent in $O(n)$, then each parent lookup is $O(1)$, giving $O(n)$ total.